SECCIÓN 2: EL PATRÓN OBSERVER Y LA PROGRAMACIÓN REACTIVA

4. Introducción al Patrón Observer

➔Librería de Project Reactor: Esta librería de Project Reactor (usa el patrón observer) **implementa** lo que es la especificación de Stream Reactivos de Java, para ser exactos.

➔Hay dos librerías de que implementan la especificación de Stream reactivos, la que es **Rxjava** y **Project Reactor**. Sin embargo, Spring está basado en la segunda.

➔Ejemplo: JPA no es implementado por Java. Java lo único que hace es que provee las interfaces necesarias para que JPA funcione. Sin embargo, son software o librerías de terceros como Hibernate o Eclipselink, las cuales hacen la implementación de estas interfaces y hacen que funcione. En el caso de Project Reactor es exactamente lo mismo.

➔Conceptos básicos:

```
Patrón Observer:
Es un patrón de diseño de comportamiento
Permite definir una dependencia uno-a-muchos entre objetos
Cuando un objeto cambia su estado, todos sus dependientes son notificados
automáticamente
Componentes principales:
 • Observable/Subject: mantiene la lista de observers y los notifica
 • Observer: interface que define cómo recibir actualizaciones
 • ConcreteObserver: implementación específica que reacciona a las
   actualizaciones
```

```
Stream (Flujo de datos):
 • Es una secuencia de elementos que pueden ser procesados
 • Características principales:
    ◦ Puede ser infinito o finito
    ◦ Puede ser síncrono o asíncrono
    ◦ Permite operaciones en pipeline (map, filter, reduce)
    ◦ No almacena datos, los procesa en tiempo real
```

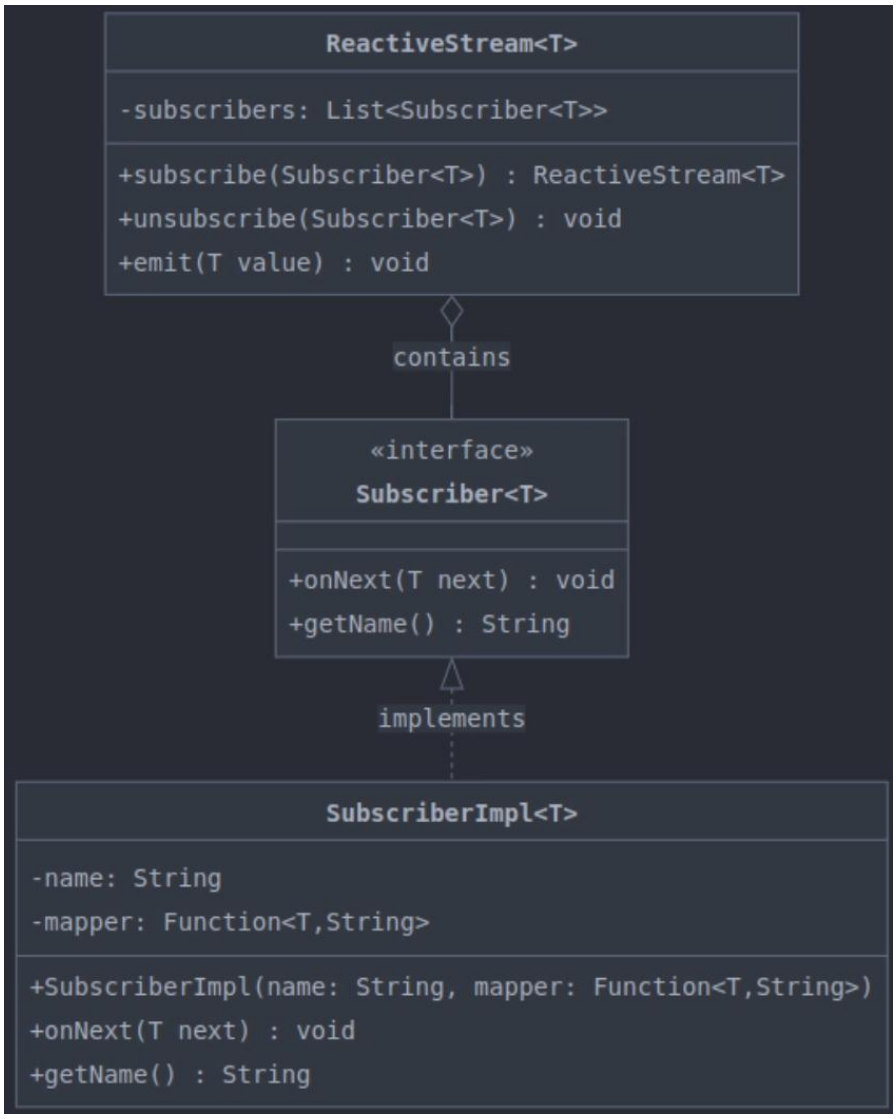
```
Publisher:
Fuente de datos que emite elementos
Controla cuándo y cómo se emiten los datos
Puede tener múltiples suscriptores
Responsabilidades:
Gestionar suscripciones
Emitir datos
Notificar errores
Señalar finalización
```

```
Subscriber(s):
Consume los datos emitidos por el Publisher
Puede procesar o transformar los datos recibidos
Operaciones típicas:
onNext(): recibe nuevo elemento
onError(): maneja errores
onComplete(): procesa finalización
Puede cancelar la suscripción cuando lo desees
```

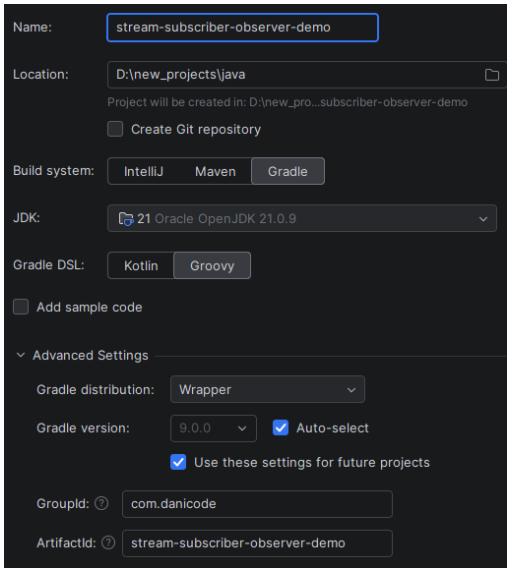
*Nota: El observable/subject es el **publisher**. Los observers son los **subscribers***

El Observable/Subject
- Es el canal de YouTube
- Puede crear y publicar nuevos videos (emit)
- Mantiene una lista de suscriptores
- Notifica a todos sus suscriptores

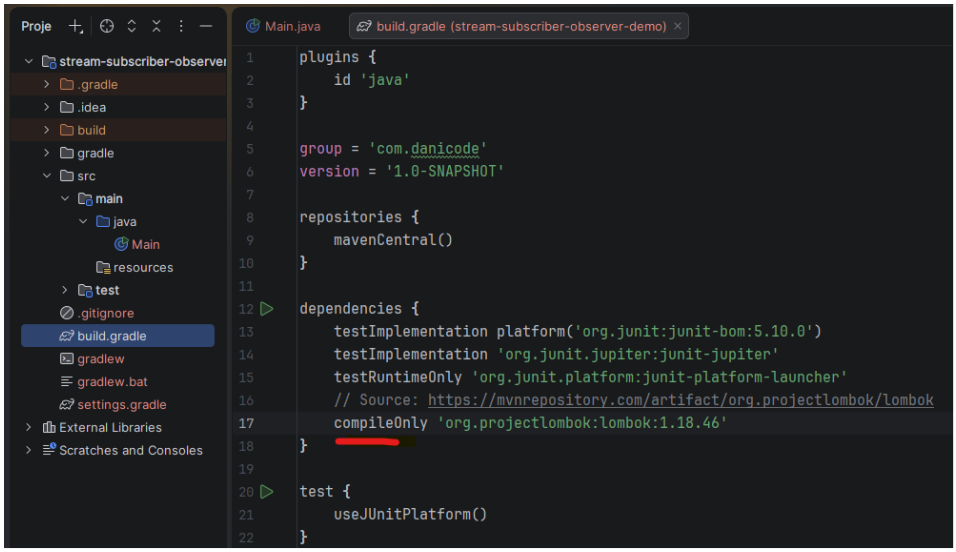
➔Diagrama de clase del patrón observer:



5. Iniciando el Proyecto

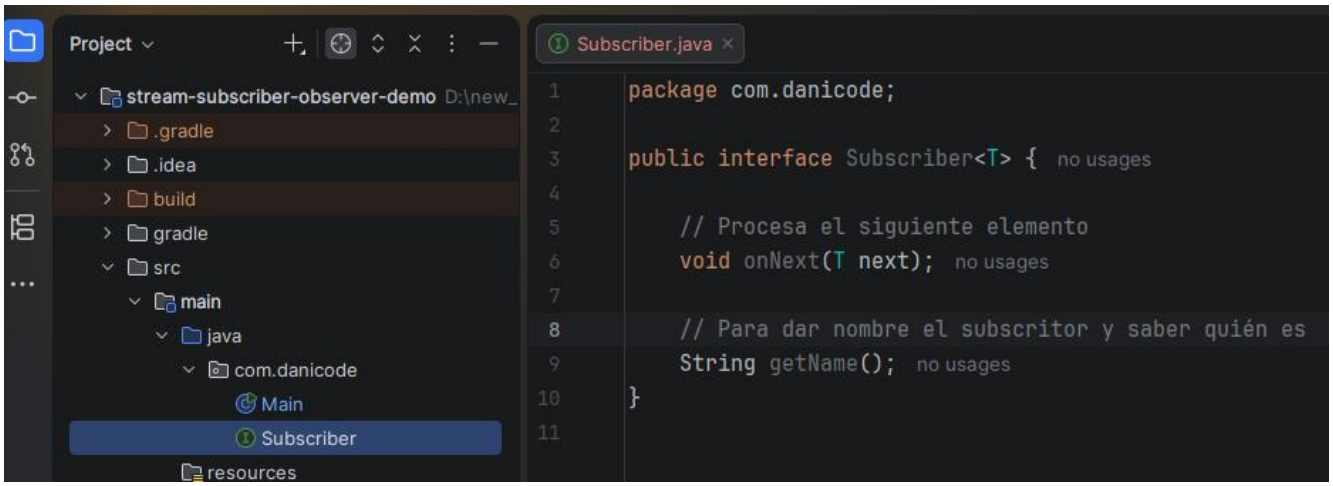


➔Agregando paquete lombok

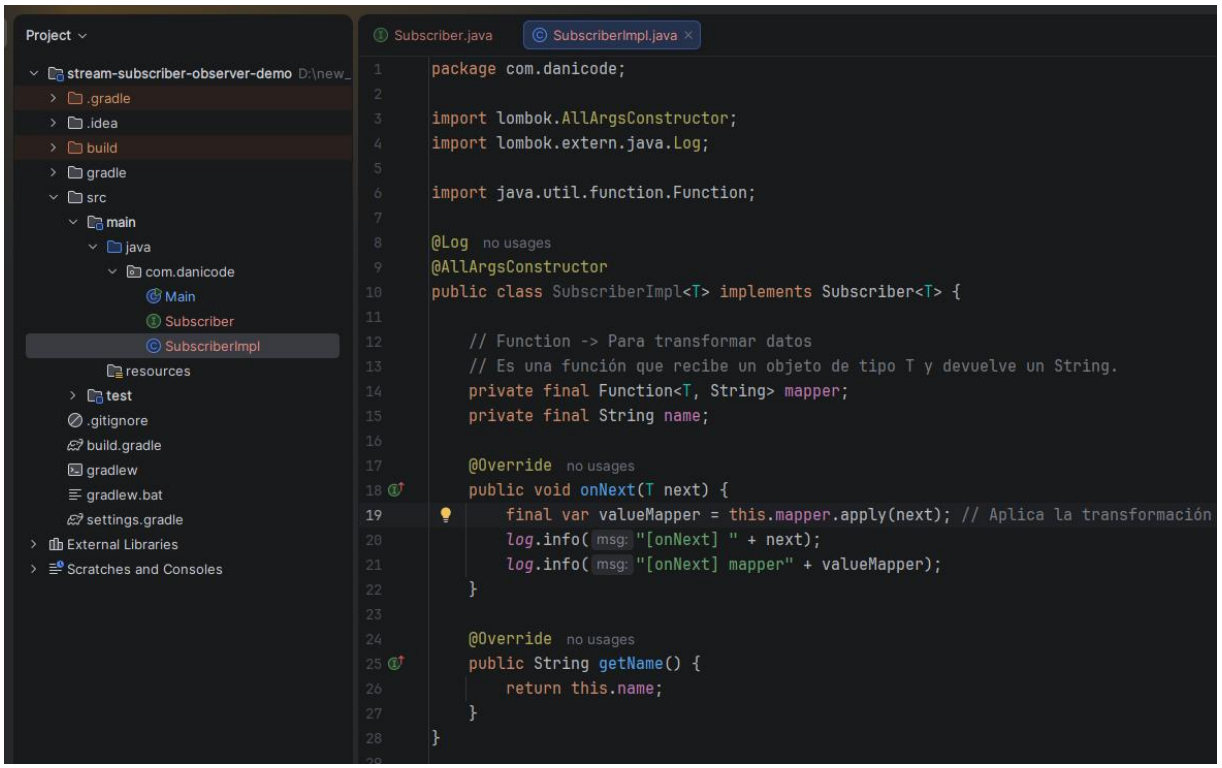


6. Implementando los Subscribers

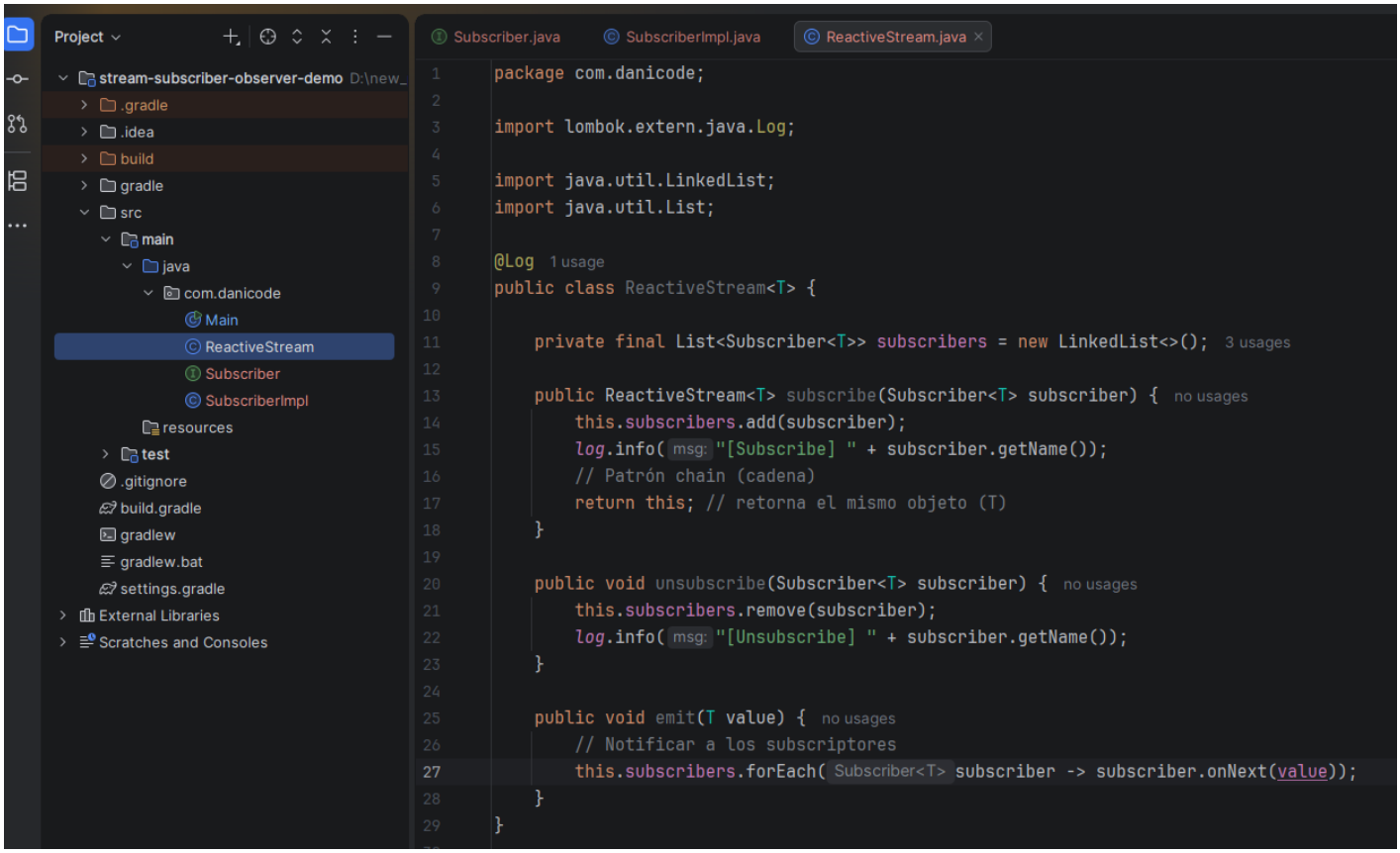
➔Creando la interfaz.



➔Implemtación:



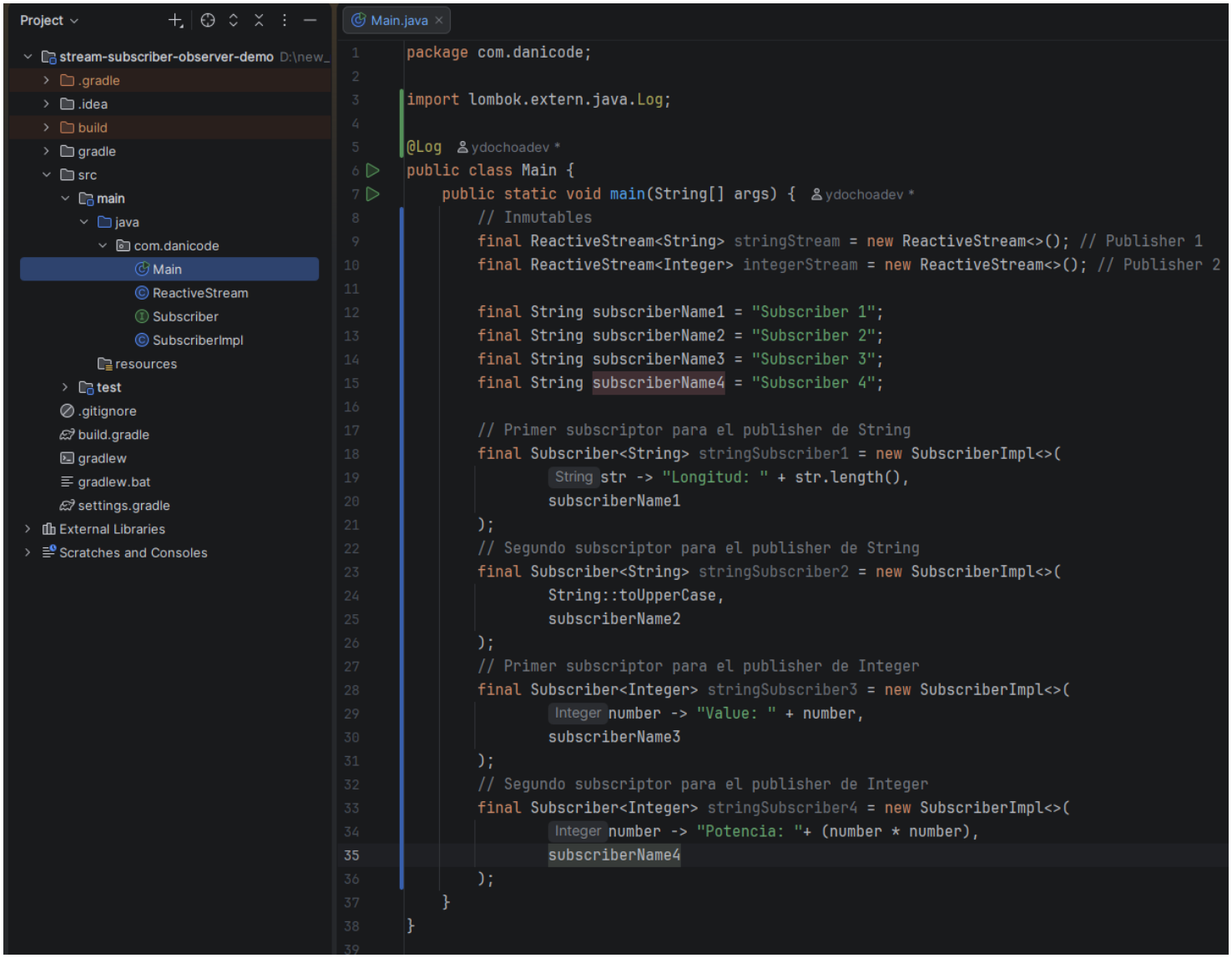
7. Implementando los Observers



Nota: ReactiveStream ➔ Publisher

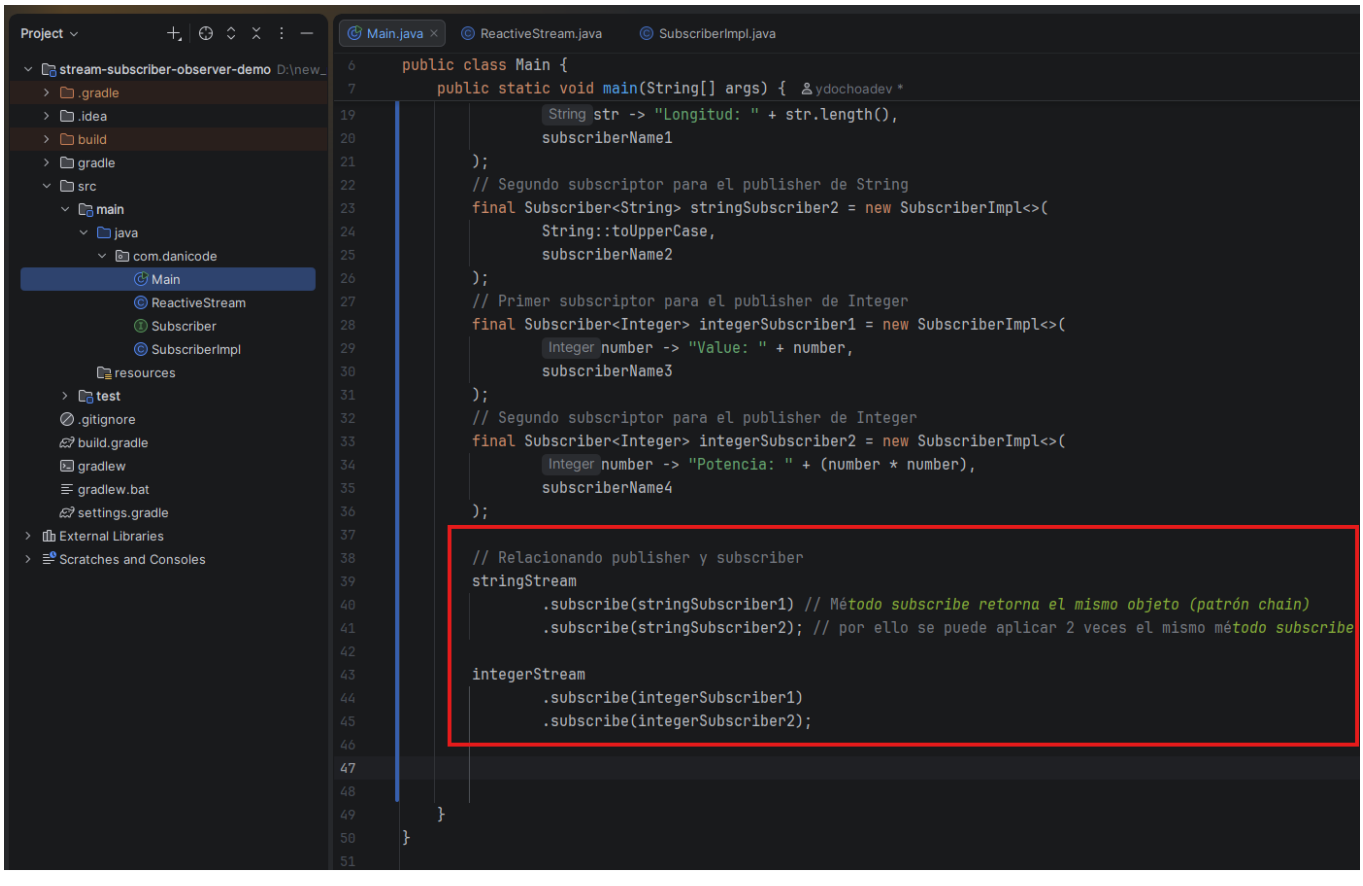
8**. Creando el Publisher-Subscriber**

➔Creando 2 publisher, uno que emita datos enteros y otro de tipo Strings. Y se crearán 2 subscriptores para cada uno.



9. Implementando Subscripciones

➔Relacionar los publicadore con sus subscriptores



➔Luego de colocar dicha relación, se debe emitir los datos, ya que el publicador aún no publica nada.

```java
// Relacionando publisher y subscriber
stringStream
        .subscribe(stringSubscriber1) // Método subscribe retorna el mismo objeto (patrón chain)
        .subscribe(stringSubscriber2); // por ello se puede aplicar 2 veces el mismo método subscribe

integerStream
        .subscribe(integerSubscriber1)
        .subscribe(integerSubscriber2);

System.out.println("--- [Strings] ---");
// Emitiendo datos
stringStream.emit( value: "hello world");
stringStream.emit( value: "this is a string");
stringStream.emit( value: "teclado, mouse y monitor");

System.out.println("--- [Numbers] ---");
integerStream.emit( value: 5);
integerStream.emit( value: 10);
integerStream.emit( value: 15);
integerStream.emit( value: 12);

// Desuscribir
stringStream.unsubscribe(stringSubscriber1);
integerStream.unsubscribe(integerSubscriber2);
```

10. Depurando el Flujo Completo

➔Para solventar el error al ejecutar (por gradle). Para que muestre el log de lombok

